

03a Baseline Time Series Models

Objective: benchmark classical baseline models for monthly CFPB credit reporting share using the trimmed 2012-01-01 to 2025-12-31 window and identify the strongest interpretable baselines for the 2026 outlook.

1. Set Up Data

- Dataset: `../data/processed/credit_reporting_monthly_data.csv`
- Scope: products whose label starts with `Credit reporting` (merged)
- Date window: `2012-01-01` to `2025-12-31`
- Frequency: monthly credit reporting share (%)
- Dependent variable: `credit_reporting_share_pct`
- Split: fixed train/validation/test with validation and test sized as evenly as possible
- Forecast horizon: 12 months

Forecasting Mechanics: Multi-Step Behavior, Error Propagation, and Uncertainty

This notebook evaluates **multi-step forecasts** on validation and test windows, and uses the same setup when producing the final 12-month forward baseline outlook.

How horizons are generated in this notebook:

- **Naive / Moving Average / Seasonal Naive / FFT** use deterministic direct `h`-step projections from the last available history.
- **ETS / ARIMA / SARIMA / Theta** are fit on the training history and then forecast an entire horizon (`h=len(val)` or `h=len(test)`) using each model's internal state recursion.

What this implies for feature construction:

- The baseline workflow is **univariate** and model-driven (no externally engineered lag-feature matrix for these classical models).
- Recursive dependence for state-space/ARIMA-family methods occurs **inside the model state updates**, not through manually re-created feature columns.

Error propagation across horizon:

- For recursive/state-space baselines, uncertainty and potential bias typically grow with forecast distance because later steps depend on earlier predicted states.

- Reported validation/test metrics are horizon-aggregated summaries for each holdout block, so they should be interpreted as **average block performance**, not equal confidence at each step.

Uncertainty treatment in this version:

- The comparison table is based on **point forecasts**.
- Statistical intervals are not yet reported in the table; therefore, confidence in long-horizon points should be treated as lower than near-horizon points.
- For stricter uncertainty quantification, add model-specific prediction intervals and/or rolling-origin backtesting by horizon in a follow-up notebook.

Implementation Specification (Reviewer Quick Reference)

This table consolidates the concrete implementation choices used in this notebook.

Component	Multi-step strategy	Horizon handling	Parameter choices in this notebook	Metrics
Naive	Direct deterministic repetition of last value	Forecast blocks use <code>h=len(val)</code> and <code>h=len(test)</code> ; outlook uses <code>h=12</code>	No tunable hyperparameters	Comparison rule as
Moving Average	Direct deterministic projection from recent mean	Same as above	<code>k=4</code> (fixed)	Same as baseline, sMAPE, runtime
Seasonal Naive	Direct deterministic seasonal repetition	Same as above	<code>season=12</code> (<code>SEASONAL_PERIODS</code>)	Same as baseline
ETS (SES / Holt-Winters)	Statsmodels forecast over full horizon from fitted state	Same as above	SES with <code>initialization_method='estimated'</code> ; Holt-Winters additive trend + additive seasonality with <code>seasonal_periods=12</code>	Same as baseline, check diagnostic section
ARIMA	Model-based horizon forecast after AIC order search	Same as above	Grid search bounds: <code>(p,d,q)</code> with <code>max_p=3, max_d=2, max_q=3</code> ; best AIC order retained	Same as baseline, comparison baseline

Component	Multi-step strategy	Horizon handling	Parameter choices in this notebook	Metrics
SARIMA	Model-based horizon forecast after seasonal AIC search	Same as above	Seasonal grid bounds: (P,D,Q,12) with max_P=2, max_D=1, max_Q=2 , conditional on selected non-seasonal order	Same as comparison baseline
Theta	Statsmodels Theta forecast over full horizon	Same as above	ThetaModel(period=12, deseasonalize=True)	Same as robustness checks 8.1-8.3
FFT	Direct frequency-reconstruction projection	Same as above	Top-frequency retention keep=6	Same as interpretation caveats

Global evaluation protocol:

- primary split is chronological train/validation/test with equalized val/test size where possible,
- primary ranking uses held-out test metrics: **sMAPE first, then MAE, then runtime**,
- robustness is stress-tested with rolling-origin validation and supplementary significance/bias/horizon diagnostics.

```
In [2]: # =====
# IMPORT STATEMENTS
# =====
# Import core libraries for time series analysis, numerical computation, and visualization

import time
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from itertools import product

# Import statsmodels forecasting and diagnostics modules
from statsmodels.tsa.holtwinters import ExponentialSmoothing, SimpleExpSmoothing
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.forecasting.theta import ThetaModel
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.stats.diagnostic import acorr_ljungbox

warnings.filterwarnings("ignore")
```

```
In [3]: # =====
# SECTION 1: DATA SETUP
# =====
# Load monthly credit reporting share series and create train/val/test splits.
```

```

import os

FREQ_ALIAS = "M"
SEASONAL_PERIODS = 12
FORECAST_HORIZON = 12

source_candidates = [
    "../data/processed/credit_reporting_monthly_data.csv",
    "../data/processed/credit_reporting_timeseries_all.parquet",
    "../archived/data/processed/credit_reporting_monthly_data.csv",
]

source_path = next((p for p in source_candidates if os.path.exists(p)), None)
if source_path is None:
    raise FileNotFoundError(f"No monthly source found. Checked: {source_candidates}")

if source_path.endswith(".parquet"):
    df = pd.read_parquet(source_path)
else:
    df = pd.read_csv(source_path)

if "month" in df.columns:
    month_col = "month"
elif "year_month" in df.columns:
    month_col = "year_month"
else:
    raise KeyError("Expected a month column named 'month' or 'year_month'.")

if "credit_reporting_share_pct" in df.columns:
    target_col = "credit_reporting_share_pct"
elif "credit_reporting_share" in df.columns:
    target_col = "credit_reporting_share"
else:
    raise KeyError("Expected target column 'credit_reporting_share_pct' or 'credit_")

df["month"] = pd.PeriodIndex(pd.to_datetime(df[month_col]), freq=FREQ_ALIAS)
df = df.sort_values("month")

series_df = (
    df[["month", target_col]]
    .rename(columns={target_col: "y"})
    .set_index("month")
    .astype(float)
)
y = series_df["y"].copy()

n = len(y)
test_size = max(4, n // 5)
val_size = test_size
while n - (val_size + test_size) < 8 and test_size > 2:
    test_size -= 1
    val_size = test_size

train = y.iloc[: n - val_size - test_size].copy()
val = y.iloc[n - val_size - test_size : n - test_size].copy()
test = y.iloc[n - test_size :].copy()

```

```
print(f"Source: {source_path}")
print(f"Frequency: {FREQ_ALIAS} | Train={len(train)}, Val={len(val)}, Test={len(test)}")
display(series_df.head())
```

Source: ../data/processed/credit_reporting_timeseries_all.parquet

Frequency: M | Train=102, Val=33, Test=33

y	
month	
2012-01	0.0
2012-02	0.0
2012-03	0.0
2012-04	0.0
2012-05	0.0

Validation Design Rationale

The initial split is intentionally designed around the forecasting objective:

- **Horizon alignment:** with a 12-month forecast horizon, both validation and test blocks are set large enough to evaluate a full-year cycle rather than only near-term points.
- **Temporal realism:** train/validation/test are contiguous chronological blocks to avoid leakage and preserve operational forecasting conditions.
- **Non-stationarity awareness:** because complaint-share dynamics can drift over time, one fixed split is treated as a primary benchmark, not a complete validation claim.
- **Multiple windows:** Section 8.2 adds rolling-origin evaluation across many forecast origins, and Section 8.3 quantifies horizon coverage plus recent-vs-earlier drift to test whether rankings are window-sensitive.

This structure balances interpretability (single benchmark split) and robustness (multi-window stress test).

2. Define Helpers

This section defines reusable utility functions and global state for the forecasting pipeline.

We implement three categories of helpers:

1. **Metric Functions** (sMAPE, MAE, RMSE): Standard evaluation metrics for comparing forecast accuracy across all models
2. **Baseline Forecast Functions** (Naive, Moving Average, Seasonal Naive, FFT): Simple, interpretable methods that serve as strong benchmarks

3. **Diagnostics Functions** (spectral entropy, residual diagnostics, plotting): Tools to assess model quality and visualize predictions

Why these three error metrics are used together for this problem:

- **MAE**: captures average absolute miss in the same units as the target share; easy to interpret as typical monthly error magnitude.
- **RMSE**: penalizes large misses more heavily than MAE, which is useful for this series because late-period level shifts can create occasional large forecast errors.
- **sMAPE**: provides a scale-normalized percentage-style view that supports fairer comparison across models and windows when the series level changes over time.

Interpretation intent in this notebook:

- model ranking uses the held-out test window with **sMAPE first**, then **MAE**, then runtime,
- MAE/RMSE are retained to distinguish stable models from those with heavy-tail error behavior,
- no single metric is treated as complete on its own; agreement across the three indicates more reliable forecast quality.

These helpers are applied consistently across all model families in Sections 3-7, enabling fair comparison.

```
In [4]: # =====
# SECTION 2: HELPER FUNCTIONS & UTILITIES
# =====

# ---- Metric Functions ----
def smape(y_true, y_pred):
    """Compute Symmetric Mean Absolute Percentage Error (sMAPE)"""
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    denom = np.abs(y_true) + np.abs(y_pred)
    ratio = np.where(denom == 0, 0.0, 2.0 * np.abs(y_true - y_pred) / denom)
    return float(100.0 * np.mean(ratio))

def mae(y_true, y_pred):
    """Compute Mean Absolute Error (MAE)"""
    return float(np.mean(np.abs(np.asarray(y_true) - np.asarray(y_pred))))

def rmse(y_true, y_pred):
    """Compute Root Mean Squared Error (RMSE)"""
    return float(np.sqrt(np.mean((np.asarray(y_true) - np.asarray(y_pred)) ** 2)))

# ---- Baseline Forecast Functions ----
def fc_naive(train_series, h):
    """Naive forecast: repeat last observed value h times"""
    return np.repeat(train_series.iloc[-1], h)

def fc_mavg(train_series, h, k=4):
```

```

        """Moving average forecast: repeat the mean of last k values h times"""
        k_eff = min(k, len(train_series))
        return np.repeat(train_series.iloc[-k_eff:].mean(), h)

def fc_snaive(train_series, h, season=SEASONAL_PERIODS):
    """Seasonal naive forecast: repeat the last seasonal cycle"""
    if len(train_series) < season:
        return fc_naive(train_series, h)
    last_season = train_series.iloc[-season:].values
    reps = int(np.ceil(h / season))
    return np.tile(last_season, reps)[:h]

def fc_fft(train_series, h, keep=6):
    """FFT-based forecast: retain top k frequency components for reconstruction"""
    values = np.asarray(train_series, dtype=float)
    n = len(values)
    x = np.arange(n, dtype=float)

    # Step 1: Remove linear trend before FFT, then add it back after reconstruction
    slope, intercept = np.polyfit(x, values, 1)
    trend = slope * x + intercept
    detr = values - trend

    # Step 2: Compute FFT and identify top k dominant frequencies
    fft_vals = np.fft.rfft(detr)
    amps = np.abs(fft_vals)
    keep = min(keep, len(amps))
    idx = np.argsort(amps)[-keep:] # Indices of largest amplitudes

    # Step 3: Filter FFT to keep only top k components
    filt = np.zeros_like(fft_vals, dtype=complex)
    filt[idx] = fft_vals[idx]

    # Step 4: Reconstruct signal into future horizon and add back trend
    x_future = np.arange(n + h, dtype=float)
    recon = np.zeros(n + h, dtype=float)
    freqs = np.fft.rfftfreq(n, d=1.0)
    for k, f in enumerate(freqs):
        c = filt[k]
        if np.abs(c) == 0:
            continue
        recon += (2.0 / n) * np.real(c * np.exp(2j * np.pi * f * x_future))

    trend_future = slope * x_future + intercept
    out = recon + trend_future
    return out[-h:] # Return only future h steps

# ---- Results & Registry Functions ----
def append_result(model_name, val_pred, test_pred, runtime_sec, fit_obj=None):
    """Store model evaluation metrics and predictions in global registry"""
    eval_rows.append({
        "Model": model_name,
        "sMAPE_val": smape(val.values, val_pred),
        "MAE_val": mae(val.values, val_pred),
        "RMSE_val": rmse(val.values, val_pred),
        "sMAPE_test": smape(test.values, test_pred),
    })

```

```

        "MAE_test": mae(test.values, test_pred),
        "RMSE_test": rmse(test.values, test_pred),
        "runtime_sec": runtime_sec
    })
    model_registry[model_name] = {
        "fit": fit_obj,
        "val_pred": pd.Series(val_pred, index=val.index, name=model_name),
        "test_pred": pd.Series(test_pred, index=test.index, name=model_name)
    }

# ---- Visualization Functions ----
def plot_section_predictions(section_title, pred_map):
    """Plot observed train/val/test series alongside model predictions for a section
    fig, ax = plt.subplots(figsize=(10, 4))

    # Convert period index to timestamp for plotting
    y_train_ts = pd.Series(train.values, index=train.index.to_timestamp(how="end"))
    y_val_ts = pd.Series(val.values, index=val.index.to_timestamp(how="end"))
    y_test_ts = pd.Series(test.values, index=test.index.to_timestamp(how="end"))

    # Plot observed data with different line styles for each period (train/val/test)
    ax.plot(y_train_ts.index, y_train_ts.values, color="steelblue", label="Observed")
    ax.plot(y_val_ts.index, y_val_ts.values, color="steelblue", linestyle="--", alpha=0.5)
    ax.plot(y_test_ts.index, y_test_ts.values, color="steelblue", linestyle=":", alpha=0.5)

    # Overlay model predictions for both val and test periods
    for model_name, preds in pred_map.items():
        val_ts = pd.Series(preds["val"], index=val.index.to_timestamp(how="end"))
        test_ts = pd.Series(preds["test"], index=test.index.to_timestamp(how="end"))
        ax.plot(val_ts.index, val_ts.values, linestyle="--", marker="o", label=f"{model_name} - Val")
        ax.plot(test_ts.index, test_ts.values, marker="o", label=f"{model_name} - Test")

    ax.set_title(section_title)
    ax.legend(ncol=2, fontsize=8)
    plt.tight_layout()
    plt.show()

# ---- Diagnostics Functions ----
def compute_spectral_entropy(series):
    """Compute normalized spectral entropy (0-1 scale; higher = less predictable)"""
    x = np.asarray(series, dtype=float)
    x = x - np.mean(x) # Center the data
    psd = np.abs(np.fft.rfft(x)) ** 2 # Power Spectral Density
    if psd.sum() == 0:
        return 0.0
    p = psd / psd.sum() # Normalize to probability distribution
    p = p[p > 0] # Remove zero probabilities for entropy calculation
    h = -np.sum(p * np.log2(p)) # Shannon entropy
    h_max = np.log2(len(psd)) if len(psd) > 1 else 1.0 # Maximum possible entropy
    return float(h / h_max) # Normalized entropy

def run_residual_diagnostics(df_results, registry, top_n=2):
    """Extract residual diagnostics (mean, std, Ljung-Box test) for top N models"""
    top_models = df_results.head(top_n)["Model"].tolist()
    rows = []
    for m in top_models:

```

```

fit_obj = registry[m]["fit"]
if fit_obj is not None and hasattr(fit_obj, "resid"):
    resid = pd.Series(fit_obj.resid).dropna()
    # Ljung-Box test for residual autocorrelation at lag 4 (seasonal)
    lb_p = float(acorr_ljungbox(resid, lags=[4], return_df=True)["lb_pvalue"])
    rows.append({
        "Model": m,
        "resid_mean": float(resid.mean()),
        "resid_std": float(resid.std(ddof=1)),
        "ljung_box_p_lag4": lb_p
    })
else:
    rows.append({
        "Model": m,
        "resid_mean": np.nan,
        "resid_std": np.nan,
        "ljung_box_p_lag4": np.nan
    })
return pd.DataFrame(rows)

# ---- Global State: Tracking Results & Models ----
eval_rows = [] # List of evaluation metric dicts (populated by each section)
model_registry = {} # Dict storing fit objects and predictions for all models

```

3. Evaluate Simple Baselines (Naive, MA, Seasonal Naive)

This section evaluates three deterministic monthly benchmarks that require no parameter fitting and therefore act as fast reference floors:

- **Naive:** repeats the last observed month
- **Moving Average (MA):** repeats the mean of the last 4 months
- **Seasonal Naive (SNaive):** repeats the last 12-month seasonal cycle

On this trimmed monthly series, these baselines are useful for establishing how much value comes from trend and seasonal structure alone. They remain interpretable, but the later surge in complaints means the flatter variants are expected to lag any model that adapts more aggressively to the post-2023 level shift.

```

In [5]: # =====
# SECTION 3: SIMPLE BASELINES
# =====
# Evaluate Naive, Moving Average, and Seasonal Naive forecasts

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
simple_pred_map = {} # Dictionary to store predictions for visualization

for name, val_pred, test_pred in [
    # Naive: repeat last value
    ("Naive", fc_naive(train, len(val)), fc_naive(train_val, len(test))),
    # Moving Average: repeat mean of last 4 values

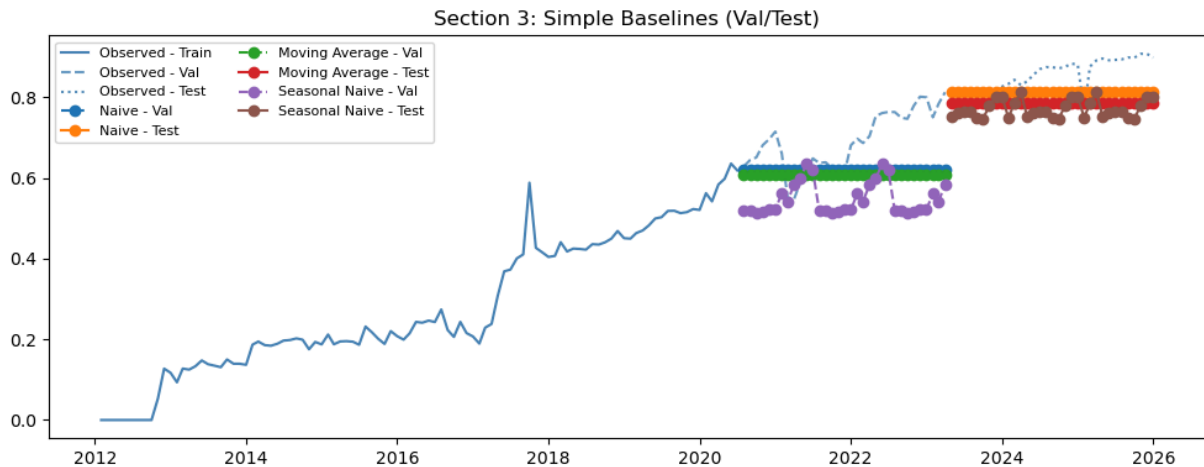
```

```

("Moving Average", fc_mavg(train, len(val), k=4), fc_mavg(train_val, len(test),
# Seasonal Naive: repeat last seasonal cycle (4 quarters)
("Seasonal Naive", fc_snaive(train, len(val), season=SEASONAL_PERIODS), fc_sna
]:
t0 = time.perf_counter()
runtime = time.perf_counter() - t0 # Simple methods are instantaneous
append_result(name, val_pred, test_pred, runtime, fit_obj=None)
simple_pred_map[name] = {"val": val_pred, "test": test_pred}

# Visualize predictions from all 3 baseline methods
plot_section_predictions("Section 3: Simple Baselines (Val/Test)", simple_pred_map)
print("Simple baselines evaluated.")

```



Simple baselines evaluated.

4. ETS (SES + Holt-Winters)

Exponential smoothing methods give more weight to recent observations, which is useful for this monthly share series because levels rise materially in the later years of the sample:

- **Simple Exponential Smoothing (SES)**: tracks the current level, but cannot represent explicit trend or yearly seasonality
- **Holt-Winters Additive**: models level, trend, and additive 12-month seasonality simultaneously

In this notebook, ETS models remain important comparators, but they are no longer the leading benchmarks after full head-to-head evaluation against Theta and FFT on the held-out windows.

```

In [6]: # =====
# SECTION 4: EXPONENTIAL SMOOTHING (ETS)
# =====
# Evaluate Simple Exponential Smoothing (SES) and Holt-Winters additive smoothing

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
ets_pred_map = {} # Dictionary to store predictions for visualization

# ---- Simple Exponential Smoothing (SES) ----

```

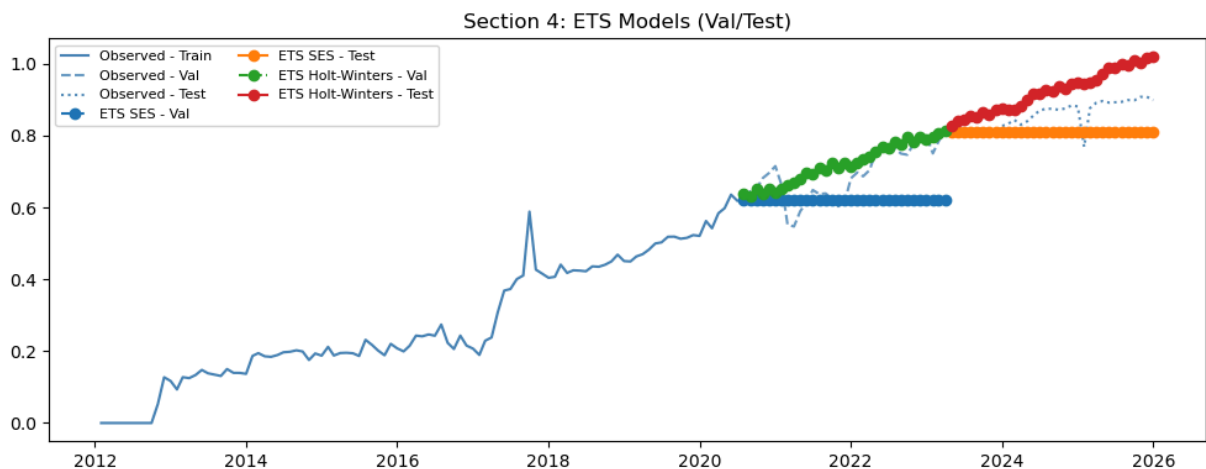
```

t0 = time.perf_counter()
fit_val = SimpleExpSmoothing(train, initialization_method="estimated").fit()
val_pred = fit_val.forecast(len(val)).values
fit_test = SimpleExpSmoothing(train_val, initialization_method="estimated").fit()
test_pred = fit_test.forecast(len(test)).values
append_result("ETS SES", val_pred, test_pred, time.perf_counter() - t0, fit_obj=fit_val)
ets_pred_map["ETS SES"] = {"val": val_pred, "test": test_pred}

# ---- Holt-Winters Additive ----
t0 = time.perf_counter()
fit_val = ExponentialSmoothing(
    train, trend="add", seasonal="add", seasonal_periods=SEASONAL_PERIODS, initial_fit=True
).fit(optimized=True)
val_pred = fit_val.forecast(len(val)).values
fit_test = ExponentialSmoothing(
    train_val, trend="add", seasonal="add", seasonal_periods=SEASONAL_PERIODS, initial_fit=True
).fit(optimized=True)
test_pred = fit_test.forecast(len(test)).values
append_result("ETS Holt-Winters", val_pred, test_pred, time.perf_counter() - t0, fit_obj=fit_val)
ets_pred_map["ETS Holt-Winters"] = {"val": val_pred, "test": test_pred}

# Visualize predictions from both ETS methods
plot_section_predictions("Section 4: ETS Models (Val/Test)", ets_pred_map)
print("ETS models evaluated.")

```



ETS models evaluated.

5. Evaluate ARIMA/SARIMA

ARIMA-family models combine autoregressive structure, differencing, and error correction to learn persistent dynamics in the monthly share series. Instead of fixing orders manually, a **stepwise AIC grid search** (implemented directly with statsmodels) selects the best parameter combination:

- **Auto ARIMA (non-seasonal)**: searches over (p, d, q) orders and picks the combination with the lowest AIC; serves as a compact non-seasonal benchmark that can absorb local trend changes

- **Auto SARIMA (seasonal)**: extends the search to seasonal (P, D, Q, m=12) parameters so the recurring within-year structure in the monthly credit reporting share series is captured explicitly

For this dataset the seasonal variant is the relevant comparator because the trimmed series still shows recurring within-year structure on top of the broader rise in share values.

```
In [7]: # =====
# SECTION 5: ARIMA & SEASONAL ARIMA (SARIMA) â€” AIC grid search
# =====
# Lightweight AIC-based grid search using statsmodels directly (no external
# dependencies). Searches (p,d,q) and (P,D,Q,m) grids and selects the order
# with the lowest AIC, then evaluates on val/test as usual.

train_val = pd.concat([train, val])
arima_pred_map = {}
arima_orders = {}

def _aic_grid_arima(series, max_p=3, max_d=2, max_q=3):
    """Return the (p,d,q) order with the lowest AIC via exhaustive grid search."""
    best_aic, best_order = np.inf, (1, 1, 1)
    for p, d, q in product(range(max_p + 1), range(max_d + 1), range(max_q + 1)):
        if p == 0 and q == 0:
            continue
        try:
            aic = ARIMA(series, order=(p, d, q)).fit().aic
            if aic < best_aic:
                best_aic, best_order = aic, (p, d, q)
        except Exception:
            pass
    return best_order

def _aic_grid_sarima(series, order, m=12, max_P=2, max_D=1, max_Q=2):
    """Return the (P,D,Q,m) seasonal order with the lowest AIC given a fixed non-se
    best_aic, best_sorder = np.inf, (1, 1, 0, m)
    for P, D, Q in product(range(max_P + 1), range(max_D + 1), range(max_Q + 1)):
        if P == 0 and Q == 0:
            continue
        try:
            aic = SARIMAX(series, order=order, seasonal_order=(P, D, Q, m)).fit(dis
            if aic < best_aic:
                best_aic, best_sorder = aic, (P, D, Q, m)
        except Exception:
            pass
    return best_sorder

# ---- Non-seasonal ARIMA (AIC grid search) ----
print("Searching non-seasonal ARIMA orders...")
t0 = time.perf_counter()
arima_order = _aic_grid_arima(train)
arima_name = f"ARIMA{arima_order}"
print(f" \u2192 Selected: {arima_name}")
fit_val = ARIMA(train, order=arima_order).fit()
val_pred = fit_val.forecast(len(val)).values
```

```

fit_test = ARIMA(train_val, order=arima_order).fit()
test_pred = fit_test.forecast(len(test)).values
append_result(arima_name, val_pred, test_pred, time.perf_counter() - t0, fit_obj=fit_test)
arima_pred_map[arima_name] = {"val": val_pred, "test": test_pred}
arima_orders[arima_name] = {"type": "arima", "order": arima_order}

# ---- Seasonal SARIMA (AIC grid search over seasonal part) ----
print("Searching seasonal SARIMA orders (this may take a moment)...")
t0 = time.perf_counter()
sarima_seasonal_order = _aic_grid_sarima(train, order=arima_order, m=SEASONAL_PERIODS)
P, D, Q, _ = sarima_seasonal_order
sarima_name = f"SARIMA{arima_order}x({P},{D},{Q},{SEASONAL_PERIODS})"
print(f" \u2192 Selected: {sarima_name}")
fit_val = SARIMAX(train, order=arima_order, seasonal_order=sarima_seasonal_order)
val_pred = fit_val.forecast(len(val)).values
fit_test = SARIMAX(train_val, order=arima_order, seasonal_order=sarima_seasonal_order)
test_pred = fit_test.forecast(len(test)).values
append_result(sarima_name, val_pred, test_pred, time.perf_counter() - t0, fit_obj=fit_test)
arima_pred_map[sarima_name] = {"val": val_pred, "test": test_pred}
arima_orders[sarima_name] = {"type": "sarima", "order": arima_order, "seasonal_order": sarima_seasonal_order}

# Visualize predictions from both ARIMA methods
plot_section_predictions("Section 5: ARIMA/SARIMA (Val/Test)", arima_pred_map)
print("ARIMA/SARIMA evaluated.")

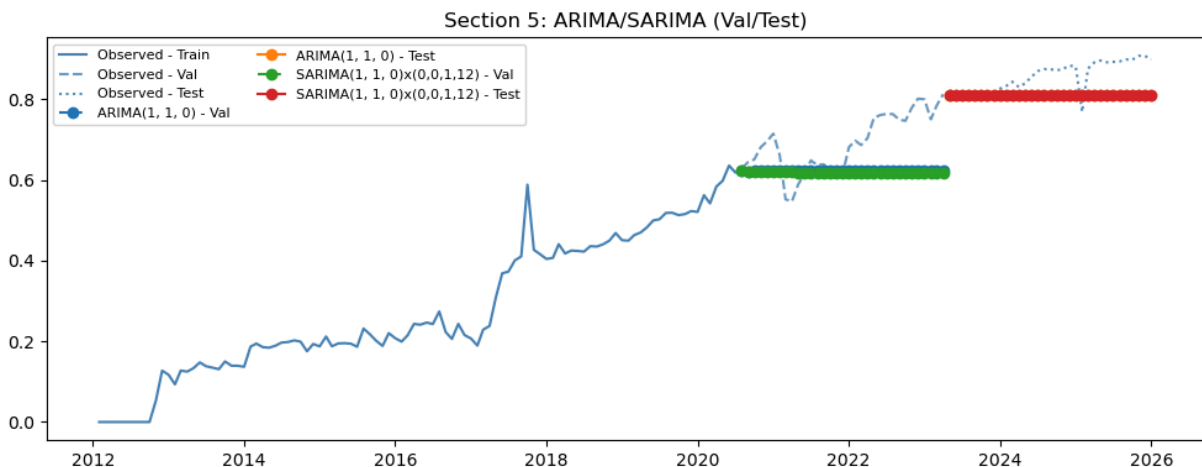
```

Searching non-seasonal ARIMA orders...

â†’ Selected: ARIMA(1, 1, 0)

Searching seasonal SARIMA orders (this may take a moment)...

â†’ Selected: SARIMA(1, 1, 0)x(0,0,1,12)



ARIMA/SARIMA evaluated.

6. Theta

The Theta method is a practical monthly forecasting baseline because it captures smooth trend behavior after deseasonalization without requiring a full seasonal state-space specification.

It works by deseasonalizing the series, fitting a Theta-style extrapolation to the adjusted signal, and then reapplying the seasonal pattern. In the current run, Theta ranks among the

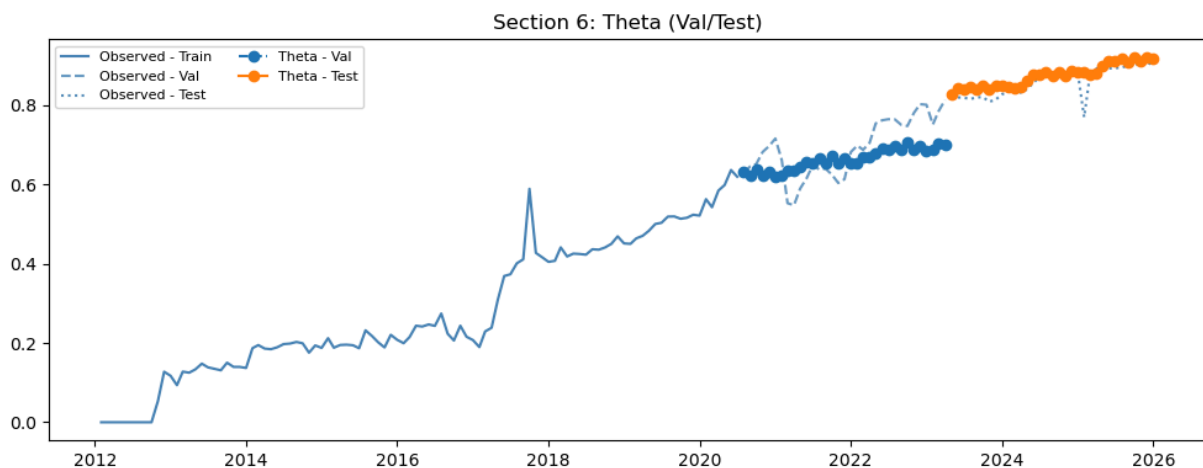
strongest models and serves as the primary interpretable benchmark for this credit reporting share series.

```
In [8]: # =====
# SECTION 6: THETA METHOD
# =====
# Evaluate Theta model with deseasonalization (effective for seasonal data)

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
theta_pred_map = {} # Dictionary to store predictions for visualization

# Theta decomposes series into Level + season before forecasting
# Works well for quarterly/seasonal data
t0 = time.perf_counter()
fit_val = ThetaModel(train, period=SEASONAL_PERIODS, deseasonalize=True).fit()
val_pred = fit_val.forecast(len(val)).values
fit_test = ThetaModel(train_val, period=SEASONAL_PERIODS, deseasonalize=True).fit()
test_pred = fit_test.forecast(len(test)).values
append_result("Theta", val_pred, test_pred, time.perf_counter() - t0, fit_obj=fit_t
theta_pred_map["Theta"] = {"val": val_pred, "test": test_pred}

# Visualize Theta predictions vs observed
plot_section_predictions("Section 6: Theta (Val/Test)", theta_pred_map)
print("Theta evaluated.")
```



7. FFT

The Fast Fourier Transform (FFT) takes a frequency-domain approach to forecasting:

1. **Detrend** the series (remove linear trend)
2. **Compute FFT** to extract the dominant frequency components
3. **Retain top 6 frequencies** (largest amplitudes) to focus on the most significant seasonal and cyclical patterns
4. **Reconstruct** the signal in the future horizon using only these key frequencies
5. **Re-trend** by adding back the linear trend component

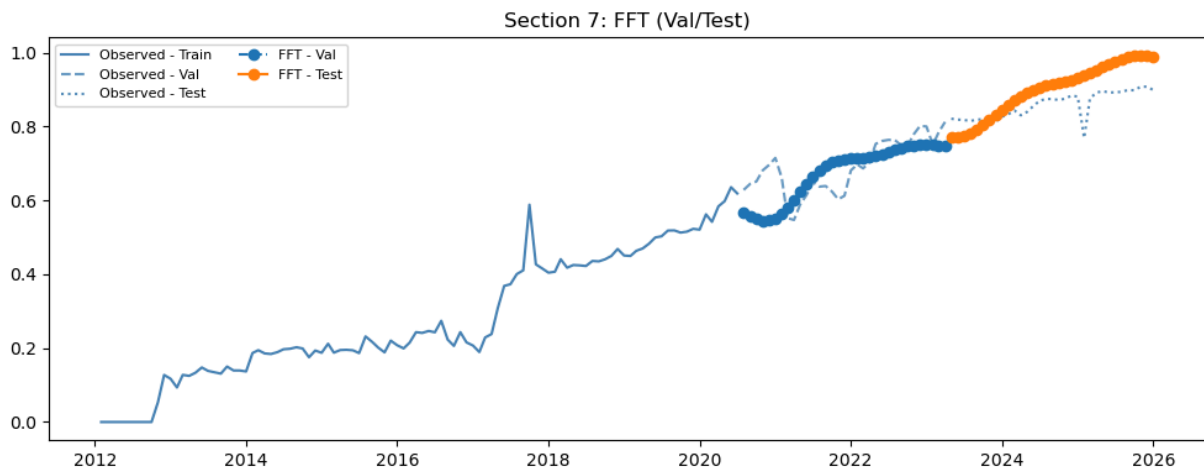
This method discovers dominant cycles empirically and is useful when the seasonal pattern is strong and stable. It complements statistical methods by exploiting periodicity without distributional assumptions.

```
In [9]: # =====
# SECTION 7: FAST FOURIER TRANSFORM (FFT)
# =====
# Evaluate FFT-based frequency reconstruction for seasonal forecasting

train_val = pd.concat([train, val]) # Combine train+val for test-set fitting
fft_pred_map = {} # Dictionary to store predictions for visualization

# FFT method: extract top 6 frequency components and reconstruct for forecast horizon
t0 = time.perf_counter()
val_pred = fc_fft(train, len(val), keep=6) # Forecast validation set
test_pred = fc_fft(train_val, len(test), keep=6) # Forecast test set
append_result("FFT", val_pred, test_pred, time.perf_counter() - t0, fit_obj=None)
fft_pred_map["FFT"] = {"val": val_pred, "test": test_pred}

# Visualize FFT predictions vs observed
plot_section_predictions("Section 7: FFT (Val/Test)", fft_pred_map)
print("FFT evaluated.")
```



FFT evaluated.

8. Comparison Table

This table ranks all monthly baselines on the held-out test window using sMAPE, MAE, RMSE, and runtime for the trimmed 2012-2025 credit reporting share series.

Metric role in this comparison:

- **sMAPE** is the primary ranking metric because it is scale-normalized for changing series levels.
- **MAE** quantifies typical absolute monthly deviation in share units.
- **RMSE** highlights models that occasionally make larger misses.

Decision-use interpretation:

- Improvements should be read **relatively**, not only as absolute values (for example, percent reduction vs Naive or vs the current best benchmark).
- A model is treated as materially better only when gains are consistent across sMAPE and MAE and not driven by a runtime outlier.
- If RMSE worsens while sMAPE improves slightly, the model may be less robust to occasional large misses and should be reviewed before deployment.

The ranking is informative beyond the top two models: flatter methods can miss regime shifts, while stronger seasonality-aware models stay materially closer to the observed monthly path.

Metric Definitions (Quick Reference)

Metric	What it captures	Practical meaning for decisions
sMAPE	Scale-normalized percentage-style error between actual and forecast	Best for primary ranking when level changes over time; lower means better relative tracking across the whole test span
MAE	Average absolute miss in target units	Interpretable as typical monthly miss size in share points; useful for operational tolerance checks
RMSE	Square-root of mean squared error (larger misses weighted more heavily)	Stress test for risk of occasional large misses; lower RMSE implies fewer costly spikes in error

Suggested interpretation bands for this notebook context (test sMAPE):

- **Strong:** < 5
- **Moderate:** 5 to 10
- **Weak:** > 10

How to convert metrics to decisions:

- Prefer models with at least a clear relative improvement vs Naive (for example, notable sMAPE and MAE reduction together).
- Break ties by choosing the model with lower RMSE (more stable under tail-error risk).
- Use runtime as a tertiary criterion after accuracy, unless deployment latency is a hard constraint.

How to read together: models with strong and consistent performance across all three metrics are preferred over models that look good on only one metric.

```
In [10]: # =====
# SECTION 8: COMPARISON TABLE
# =====
# Aggregate all model results, add interpretive notes, and rank by test performance
```

```

# Define interpretive notes for each model based on its expected behavior
notes_map = {
    "Naive": "Too biased",
    "Moving Average": "Too flat",
    "Seasonal Naive": "Misses trend/regime shifts",
    "ETS SES": "May underfit trend",
    "ETS Holt-Winters": "Can over/under-smooth peaks",
    arima_name: "No explicit seasonality (auto-selected orders)",
    sarima_name: "Strong seasonal baseline (auto-selected orders)",
    "Theta": "Strong trend-seasonality balance",
    "FFT": "Seasonal frequency focus"
}

# Convert eval_rows list to DataFrame and attach notes
results = pd.DataFrame(eval_rows)
results["Notes"] = results["Model"].map(notes_map).fillna("")

# Sort by test performance (primary: sMAPE_test, secondary: MAE_test, tertiary: run
results = results.sort_values(["sMAPE_test", "MAE_test", "runtime_sec"]).reset_index()

# Select columns for display and round metrics to 3 decimal places
comparison_cols = ["Model", "sMAPE_val", "MAE_val", "RMSE_val", "sMAPE_test", "MAE_test", "RMSE_test"]
results_display = results[comparison_cols].copy()
for c in ["sMAPE_val", "MAE_val", "RMSE_val", "sMAPE_test", "MAE_test", "RMSE_test"]:
    results_display[c] = results_display[c].round(3)

# Export baseline comparison artifact for downstream notebooks (e.g., 02b)
artifact_dir = "../reports/artifacts"
os.makedirs(artifact_dir, exist_ok=True)
baseline_artifact_path = f"{artifact_dir}/02a_creditreporting_monthly_baseline_results.csv"
results.to_csv(baseline_artifact_path, index=False)
print(f"Saved 02a baseline artifact: {baseline_artifact_path}")

# Display the comparison table
display(results_display)

```

Saved 02a baseline artifact: ../reports/artifacts/02a_creditreporting_monthly_baseline_results.csv

	Model	sMAPE_val	MAE_val	RMSE_val	sMAPE_test	MAE_test	RMSE_test	runtime_s
0	Theta	8.203	0.056	0.064	2.011	0.017	0.026	0.0
1	Naive	12.311	0.083	0.102	5.727	0.048	0.058	0.0
2	ARIMA(1, 1, 0)	11.998	0.081	0.100	5.990	0.051	0.060	4.7
3	SARIMA(1, 1, 0)x(0,0,1,12)	12.311	0.083	0.103	6.016	0.051	0.060	7.1
4	ETS SES	12.142	0.082	0.101	6.075	0.051	0.061	0.0
5	FFT	8.356	0.055	0.069	6.160	0.055	0.063	0.0
6	ETS Holt-Winters	6.522	0.044	0.057	7.311	0.066	0.074	0.1
7	Moving Average	13.408	0.090	0.109	8.697	0.072	0.080	0.0
8	Seasonal Naive	23.195	0.146	0.164	10.594	0.087	0.095	0.0

8.1.1. Error-Strength Context (How Strong Are These Errors?)

Raw error values (MAE, RMSE, sMAPE) are useful but hard to judge in isolation. This section adds context using three complementary views:

1. Skill versus Naive baseline

- converts raw errors into relative improvement using

$$\text{Skill} = 1 - \frac{\text{Model Error}}{\text{Naive Error}}$$

- positive values indicate improvement over a simple last-value benchmark.

2. Scale-normalized absolute error

- reports NMAE (MAE normalized by the observed test-window range and mean),
- helps interpret practical miss size relative to observed variation.

3. Strength flags for interpretation

- combines sMAPE band, skill-vs-naive, and normalized MAE into a concise quality label,
- avoids over-interpreting small numerical differences as meaningful gains.

```
In [ ]: # =====
# SECTION 8.1.1: METRIC CONTEXT AND PERFORMANCE STRENGTH FLAGS
# =====

# Require standard objects from previous sections.
if len(results) == 0:
    print('No model results found for metric-context analysis.')
else:
    # Baseline references.
    if 'Naive' in results['Model'].values:
        naive_row = results[results['Model'] == 'Naive'].iloc[0]
    else:
        naive_row = results.iloc[0]

    champion_row = results.iloc[0]

    # Build contextualized score table for all models.
    ctx = results[['Model', 'sMAPE_test', 'MAE_test', 'RMSE_test']].copy()

    # Skill vs Naive: positive means better than Naive.
    ctx['Skill_vs_Naive_sMAPE'] = 1.0 - (ctx['sMAPE_test'] / max(float(naive_row['s
    ctx['Skill_vs_Naive_MAE'] = 1.0 - (ctx['MAE_test'] / max(float(naive_row['MAE_t
    ctx['Skill_vs_Naive_RMSE'] = 1.0 - (ctx['RMSE_test'] / max(float(naive_row['RMS

    # Normalized MAE context from observed test scale.
    y_test = test.values.astype(float)
    y_range = float(np.max(y_test) - np.min(y_test)) if len(y_test) else np.nan
    y_mean = float(np.mean(np.abs(y_test))) if len(y_test) else np.nan

    ctx['NMAE_by_TestRange'] = ctx['MAE_test'] / max(y_range, 1e-12)
    ctx['NMAE_by_TestMean'] = ctx['MAE_test'] / max(y_mean, 1e-12)

    # Human-readable strength label.
    def classify_strength(smape_val, skill_mae, nmae_range):
        if (smape_val < 5.0) and (skill_mae >= 0.30) and (nmae_range <= 0.10):
            return 'Strong'
        if (smape_val < 10.0) and (skill_mae >= 0.10) and (nmae_range <= 0.20):
            return 'Moderate'
        return 'Weak/Conditional'

    ctx['StrengthLabel'] = [
```

```

        classify_strength(s, k, n)
    for s, k, n in zip(ctx['sMAPE_test'], ctx['Skill_vs_Naive_MAE'], ctx['NMAE_
]

# Rounded display table.
ctx_display = ctx.copy()
pct_cols = ['Skill_vs_Naive_sMAPE', 'Skill_vs_Naive_MAE', 'Skill_vs_Naive_RMSE']
for c in pct_cols:
    ctx_display[c] = (100.0 * ctx_display[c]).round(2)

for c in ['sMAPE_test', 'MAE_test', 'RMSE_test', 'NMAE_by_TestRange', 'NMAE_by_
    ctx_display[c] = ctx_display[c].round(4)

print('Contextualized error table (test window):')
display(ctx_display)

# Champion-focused interpretation.
ch = ctx[ctx['Model'] == champion_row['Model']].iloc[0]
print('Champion context summary:')
print(
    f"- Champion: {champion_row['Model']} | "
    f"sMAPE={champion_row['sMAPE_test']:.3f}, MAE={champion_row['MAE_test']:.4f}
)
print(
    f"- Skill vs Naive (MAE): {100.0 * ch['Skill_vs_Naive_MAE']:.2f}% | "
    f"NMAE/TestRange: {ch['NMAE_by_TestRange']:.4f} | NMAE/TestMean: {ch['NMAE_
)
print(f"- Strength label: {ch['StrengthLabel']}")

if ch['StrengthLabel'] == 'Strong':
    print('Interpretation: model performance is meaningfully strong given both
elif ch['StrengthLabel'] == 'Moderate':
    print('Interpretation: model performance is credible but should be reported
else:
    print('Interpretation: model performance is weak/conditional; avoid strong

```

8.1. Practical Significance and Forecast-Bias Audit

Selecting a "best" model only from aggregate error ranks can be misleading if metric gaps are small or if a model is systematically biased (persistent over- or under-forecasting).

This section adds two checks:

1. Meaningfulness of the top-vs-runner gap

- computes relative improvement of the top model versus runner-up,
- uses bootstrap resampling of the test window to estimate a confidence interval for the MAE gap.

2. Directional forecast-bias diagnostics

- reports mean signed error (MFE),

- reports over-forecast and under-forecast rates,
- checks whether the primary model is winning by balanced errors or by biased behavior.

Decision guidance:

- treat differences as weak when gap sizes are very small and uncertainty overlaps,
- prefer models that are both accurate and directionally balanced unless a specific business cost favors one-sided bias.

```
In [12]: # =====
# SECTION 8.1: PRACTICAL SIGNIFICANCE + FORECAST-BIAS AUDIT
# =====

# Require at least two ranked models for gap analysis.
if len(results) < 2:
    print("Not enough models to run significance/bias audit.")
else:
    champion = results.iloc[0]["Model"]
    runner_up = results.iloc[1]["Model"]

    # Relative gap summary (positive values mean champion is better).
    rel_gain_smape = 100.0 * (results.iloc[1]["sMAPE_test"] - results.iloc[0]["sMAPE_test"])
    rel_gain_mae = 100.0 * (results.iloc[1]["MAE_test"] - results.iloc[0]["MAE_test"])
    rel_gain_rmse = 100.0 * (results.iloc[1]["RMSE_test"] - results.iloc[0]["RMSE_test"])

    gap_tbl = pd.DataFrame([
        {
            "Champion": champion,
            "RunnerUp": runner_up,
            "RelativeGain_sMAPE_%": rel_gain_smape,
            "RelativeGain_MAE_%": rel_gain_mae,
            "RelativeGain_RMSE_%": rel_gain_rmse,
        }
    ]).round(3)

    # Pull test predictions from the model registry for uncertainty + bias checks.
    y_true = test.values.astype(float)
    p_ch = model_registry[champion]["test_pred"].values.astype(float)
    p_ru = model_registry[runner_up]["test_pred"].values.astype(float)

    # Bootstrap CI for MAE gap: (runner-up MAE - champion MAE).
    # Positive interval suggests champion improvement is robust.
    rng = np.random.default_rng(42)
    n_test = len(y_true)
    B = 2000
    delta_mae = np.empty(B, dtype=float)

    idx = np.arange(n_test)
    for b in range(B):
        bidx = rng.choice(idx, size=n_test, replace=True)
        delta_mae[b] = mae(y_true[bidx], p_ru[bidx]) - mae(y_true[bidx], p_ch[bidx])

    ci_lo, ci_med, ci_hi = np.percentile(delta_mae, [2.5, 50, 97.5])
    signif_tbl = pd.DataFrame([
```

```

    {
        "Metric": "RunnerUp_MAE - Champion_MAE",
        "Bootstrap_CI_2.5%": ci_lo,
        "Bootstrap_Median": ci_med,
        "Bootstrap_CI_97.5%": ci_hi,
    }
]).round(4)

# Directional bias diagnostics for top 3 models (or fewer if unavailable).
top_k = min(3, len(results))
bias_rows = []
for m in results.head(top_k)["Model"].tolist():
    pred = model_registry[m]["test_pred"].values.astype(float)
    err = pred - y_true # Positive = over-forecast; Negative = under-forecast
    bias_rows.append({
        "Model": m,
        "MFE_pred_minus_actual": float(np.mean(err)),
        "OverForecastRate": float(np.mean(err > 0)),
        "UnderForecastRate": float(np.mean(err < 0)),
        "MeanAbsError": float(np.mean(np.abs(err))),
    })

bias_tbl = pd.DataFrame(bias_rows)
for c in ["MFE_pred_minus_actual", "OverForecastRate", "UnderForecastRate", "Me
    bias_tbl[c] = bias_tbl[c].round(4)

print("Top-vs-Runner Relative Gain (test window)")
display(gap_tbl)

print("Bootstrap Uncertainty for MAE Gap (runner-up minus champion)")
display(signif_tbl)

print("Directional Bias Audit (test window, top models)")
display(bias_tbl)

# Practical interpretation flags.
weak_gap = rel_gain_smape < 2.0 and rel_gain_mae < 2.0
ci_crosses_zero = (ci_lo <= 0.0 <= ci_hi)
if weak_gap or ci_crosses_zero:
    print("Interpretation: top-vs-runner differences are modest and/or uncertain
else:
    print("Interpretation: champion retains a meaningful MAE edge over runner-u

ch_bias = bias_tbl[bias_tbl["Model"] == champion].iloc[0]
if abs(ch_bias["MFE_pred_minus_actual"]) > 0.01:
    print("Bias note: champion shows directional bias; review whether over/unde
else:
    print("Bias note: champion is directionally close to balanced on average te

```

Top-vs-Runner Relative Gain (test window)

	Champion	RunnerUp	RelativeGain_sMAPE_%	RelativeGain_MAE_%	RelativeGain_RMSE_%
0	Theta	Naive	64.881	64.71	55.161

Bootstrap Uncertainty for MAE Gap (runner-up minus champion)

	Metric	Bootstrap_CI_2.5%	Bootstrap_Median	Bootstrap_CI_97.5%
0	RunnerUp_MAE - Champion_MAE	0.017	0.0314	0.0449

Directional Bias Audit (test window, top models)

	Model	MFE_pred_minus_actual	OverForecastRate	UnderForecastRate	MeanAbsError
0	Theta	0.0161	0.8788	0.1212	0.0171
1	Naive	-0.0455	0.0606	0.9394	0.0485
2	ARIMA(1, 1, 0)	-0.0480	0.0606	0.9394	0.0506

Interpretation: champion retains a meaningful MAE edge over runner-up in bootstrap comparison.

Bias note: champion shows directional bias; review whether over/under-forecasting is acceptable for downstream decisions.

8.1.2. Pairwise Significance Across Top Models

Section 8.1 tests only champion vs runner-up. This section extends the comparison to the top-ranked models to distinguish:

- **practical significance:** relative MAE/sMAPE gains in percentage terms,
- **statistical significance:** bootstrap confidence interval for paired MAE-gap on the same test months.

Interpretation rule:

- if the MAE-gap CI is entirely above zero, the row model is significantly better than the column model,
- if CI overlaps zero, differences are treated as uncertain even when average errors differ.

```
In [11]: # =====
# SECTION 8.1.2: PAIRWISE SIGNIFICANCE FOR TOP MODELS
# =====

if len(results) < 2:
    print('Not enough models for pairwise significance checks.')
else:
    top_n = min(5, len(results))
    top_models = results.head(top_n)['Model'].tolist()

    # Collect per-model test predictions.
    pred_map = {}
    for m in top_models:
        if m in model_registry and 'test_pred' in model_registry[m]:
            pred_map[m] = model_registry[m]['test_pred'].values.astype(float)

    y_true = test.values.astype(float)
```

```

available_models = [m for m in top_models if m in pred_map]

if len(available_models) < 2:
    print('Insufficient prediction vectors for pairwise significance matrix.')
else:
    B = 2000
    rng = np.random.default_rng(123)
    idx = np.arange(len(y_true))

    rows = []
    for i, m_a in enumerate(available_models):
        for j, m_b in enumerate(available_models):
            if i >= j:
                continue

            p_a = pred_map[m_a]
            p_b = pred_map[m_b]

            # Delta defined as MAE(model B) - MAE(model A); positive => A better
            delta = np.empty(B, dtype=float)
            for b in range(B):
                bidx = rng.choice(idx, size=len(idx), replace=True)
                delta[b] = mae(y_true[bidx], p_b[bidx]) - mae(y_true[bidx], p_a[bidx])

            ci_lo, ci_med, ci_hi = np.percentile(delta, [2.5, 50.0, 97.5])

            mae_a = mae(y_true, p_a)
            mae_b = mae(y_true, p_b)
            smape_a = smape(y_true, p_a)
            smape_b = smape(y_true, p_b)

            rel_gain_mae = 100.0 * (mae_b - mae_a) / max(mae_b, 1e-12)
            rel_gain_smape = 100.0 * (smape_b - smape_a) / max(smape_b, 1e-12)

            sig_label = 'Significant (A better)' if ci_lo > 0 else ('Significant (B better)' if ci_hi < 0 else 'Not Significant')

            rows.append({
                'Model_A': m_a,
                'Model_B': m_b,
                'RelGain_MAE_A_vs_B_%': rel_gain_mae,
                'RelGain_sMAPE_A_vs_B_%': rel_gain_smape,
                'Bootstrap_MAEgap_CI_2.5%': ci_lo,
                'Bootstrap_MAEgap_Median': ci_med,
                'Bootstrap_MAEgap_CI_97.5%': ci_hi,
                'Decision': sig_label,
            })

    pairwise_sig = pd.DataFrame(rows)
    num_cols = [
        'RelGain_MAE_A_vs_B_%',
        'RelGain_sMAPE_A_vs_B_%',
        'Bootstrap_MAEgap_CI_2.5%',
        'Bootstrap_MAEgap_Median',
        'Bootstrap_MAEgap_CI_97.5%'
    ]
    for c in num_cols:

```

```

pairwise_sig[c] = pairwise_sig[c].round(4)

print('Pairwise practical + statistical significance among top models:')
display(pairwise_sig)

n_sig = int((pairwise_sig['Decision'] != 'Uncertain').sum())
print(f'Interpretation: {n_sig} of {len(pairwise_sig)} top-model pair compa

```

Pairwise practical + statistical significance among top models:

	Model_A	Model_B	RelGain_MAE_A_vs_B_%	RelGain_sMAPE_A_vs_B_%	Bootstrap_MAE
0	Theta	Naive	64.7101	64.8811	
1	Theta	ARIMA(1, 1, 0)	66.1967	66.4193	
2	Theta	SARIMA(1, 1, 0)x(0,0,1,12)	66.3401	66.5676	
3	Theta	ETS SES	66.6512	66.8893	
4	Naive	ARIMA(1, 1, 0)	4.2126	4.3799	
5	Naive	SARIMA(1, 1, 0)x(0,0,1,12)	4.6191	4.8022	
6	Naive	ETS SES	5.5004	5.7182	
7	ARIMA(1, 1, 0)	SARIMA(1, 1, 0)x(0,0,1,12)	0.4243	0.4417	
8	ARIMA(1, 1, 0)	ETS SES	1.3444	1.3997	
9	SARIMA(1, 1, 0)x(0,0,1,12)	ETS SES	0.9240	0.9622	

Interpretation: 10 of 10 top-model pair comparisons show non-overlapping bootstrap MAE-gap intervals.

8.1.3. Step-Ahead Horizon Profile (h=1..12)

Aggregate test metrics can hide where models degrade across lead time.

This section evaluates step-ahead performance for the top baselines by:

- computing pointwise step-ahead errors from the test forecast path,
- summarizing horizon-wise sMAPE/absolute error for `h=1` to `h=12`,

- quantifying degradation relative to `h=1` .

Interpretation:

- larger positive deltas at higher `h` indicate worsening long-horizon reliability,
- flatter profiles indicate more stable multi-step behavior.

```
In [12]: # =====
# SECTION 8.1.3: STEP-AHEAD HORIZON ERROR PROFILE
# =====

if len(results) == 0 or 'model_registry' not in globals():
    print('Results/model registry not available. Run Sections 3-8 first.')
else:
    top_n = min(3, len(results))
    top_models = results.head(top_n)['Model'].tolist()

    h_cap = min(int(FORECAST_HORIZON), len(test))
    y_true = test.values.astype(float)[:h_cap]
    step_rows = []

    for model_name in top_models:
        if model_name not in model_registry or 'test_pred' not in model_registry[model_name]:
            continue

        p = model_registry[model_name]['test_pred'].values.astype(float)[:h_cap]

        for h in range(h_cap):
            yt = float(y_true[h])
            yp = float(p[h])
            abs_err = abs(yt - yp)
            smape_pt = 100.0 * (2.0 * abs_err / max(abs(yt) + abs(yp), 1e-12))

            step_rows.append({
                'Model': model_name,
                'Horizon': h + 1,
                'sMAPE_step': smape_pt,
                'AbsError_step': abs_err,
            })

    if len(step_rows) == 0:
        print('No step-ahead predictions found for top models.')
    else:
        horizon_step = pd.DataFrame(step_rows)
        horizon_step = horizon_step.sort_values(['Model', 'Horizon']).reset_index(drop=True)

        # Degradation vs h=1 for each model.
        for c in ['sMAPE_step', 'AbsError_step']:
            base = horizon_step.groupby('Model')[c].transform('first')
            horizon_step[f'{c}_delta_vs_h1'] = horizon_step[c] - base

        horizon_step_report = horizon_step.copy()
        for c in ['sMAPE_step', 'AbsError_step', 'sMAPE_step_delta_vs_h1', 'AbsError_step_delta_vs_h1']:
            horizon_step_report[c] = horizon_step_report[c].round(4)
```

```

print('Step-ahead performance profile for top models (h=1..12):')
display(horizon_step_report)

# Slope summary on first 12 test steps.
slope_rows = []
for model_name, g in horizon_step.groupby('Model'):
    x = g['Horizon'].values.astype(float)
    sm_slope = float(np.polyfit(x, g['sMAPE_step'].values.astype(float), 1))
    ae_slope = float(np.polyfit(x, g['AbsError_step'].values.astype(float), 1))
    slope_rows.append({
        'Model': model_name,
        'sMAPE_Slope_per_Horizon': sm_slope,
        'AbsError_Slope_per_Horizon': ae_slope,
    })

horizon_step_slopes = pd.DataFrame(slope_rows)
for c in ['sMAPE_Slope_per_Horizon', 'AbsError_Slope_per_Horizon']:
    horizon_step_slopes[c] = horizon_step_slopes[c].round(4)

print('Horizon degradation slopes (first 12 test steps):')
display(horizon_step_slopes)

# Visual profile.
fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharex=True)
for model_name, g in horizon_step.groupby('Model'):
    axes[0].plot(g['Horizon'], g['sMAPE_step'], marker='o', label=model_name)
    axes[1].plot(g['Horizon'], g['AbsError_step'], marker='o', label=model_name)

axes[0].set_title('Step-ahead sMAPE by Horizon')
axes[0].set_xlabel('Horizon (steps ahead)')
axes[0].set_ylabel('sMAPE')
axes[0].grid(True, alpha=0.3)

axes[1].set_title('Step-ahead Absolute Error by Horizon')
axes[1].set_xlabel('Horizon (steps ahead)')
axes[1].set_ylabel('Absolute Error')
axes[1].grid(True, alpha=0.3)

handles, labels = axes[0].get_legend_handles_labels()
if len(handles) > 0:
    fig.legend(handles, labels, loc='upper center', ncol=max(1, len(labels)))

plt.tight_layout(rect=[0, 0, 1, 0.9])
plt.show()

worst = horizon_step.sort_values('sMAPE_step', ascending=False).iloc[0]
print(
    f"Interpretation: largest step-ahead miss among top models occurs at h= {worst['Horizon']}"
    f"for {worst['Model']} (sMAPE={worst['sMAPE_step']:.3f})."
)

```

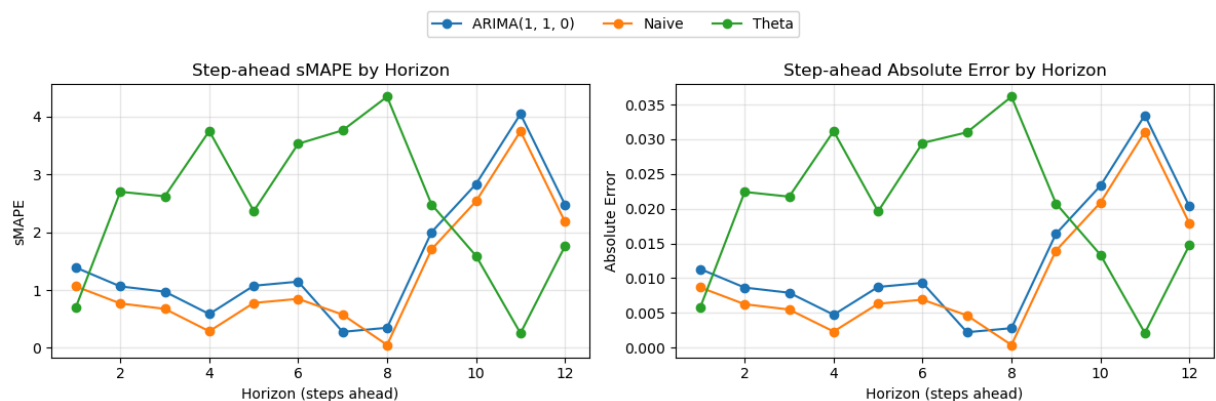
Step-ahead performance profile for top models (h=1..12):

	Model	Horizon	sMAPE_step	AbsError_step	sMAPE_step_delta_vs_h1	AbsError_step_delta_vs_h1
0	ARIMA(1, 1, 0)	1	1.3874	0.0113	0.0000	
1	ARIMA(1, 1, 0)	2	1.0614	0.0086	-0.3260	
2	ARIMA(1, 1, 0)	3	0.9708	0.0079	-0.4166	
3	ARIMA(1, 1, 0)	4	0.5830	0.0047	-0.8044	
4	ARIMA(1, 1, 0)	5	1.0728	0.0087	-0.3146	
5	ARIMA(1, 1, 0)	6	1.1439	0.0093	-0.2435	
6	ARIMA(1, 1, 0)	7	0.2744	0.0022	-1.1130	
7	ARIMA(1, 1, 0)	8	0.3455	0.0028	-1.0419	
8	ARIMA(1, 1, 0)	9	2.0009	0.0164	0.6135	
9	ARIMA(1, 1, 0)	10	2.8335	0.0233	1.4461	
10	ARIMA(1, 1, 0)	11	4.0453	0.0335	2.6579	
11	ARIMA(1, 1, 0)	12	2.4803	0.0203	1.0929	
12	Naive	1	1.0613	0.0087	0.0000	
13	Naive	2	0.7660	0.0062	-0.2953	
14	Naive	3	0.6726	0.0055	-0.3888	
15	Naive	4	0.2850	0.0023	-0.7763	
16	Naive	5	0.7747	0.0063	-0.2866	
17	Naive	6	0.8459	0.0069	-0.2155	
18	Naive	7	0.5725	0.0046	-0.4889	
19	Naive	8	0.0475	0.0004	-1.0138	
20	Naive	9	1.7029	0.0140	0.6415	
21	Naive	10	2.5356	0.0209	1.4742	
22	Naive	11	3.7474	0.0310	2.6861	
23	Naive	12	2.1823	0.0179	1.1209	

	Model	Horizon	sMAPE_step	AbsError_step	sMAPE_step_delta_vs_h1	AbsError_step_delta_vs_h1
24	Theta	1	0.7032	0.0058	0.0000	
25	Theta	2	2.6999	0.0224	1.9967	
26	Theta	3	2.6218	0.0217	1.9186	
27	Theta	4	3.7519	0.0312	3.0487	
28	Theta	5	2.3660	0.0196	1.6628	
29	Theta	6	3.5307	0.0295	2.8275	
30	Theta	7	3.7635	0.0310	3.0603	
31	Theta	8	4.3458	0.0361	3.6426	
32	Theta	9	2.4805	0.0208	1.7773	
33	Theta	10	1.5907	0.0134	0.8875	
34	Theta	11	0.2512	0.0021	-0.4520	
35	Theta	12	1.7577	0.0147	1.0545	

Horizon degradation slopes (first 12 test steps):

	Model	sMAPE_Slope_per_Horizon	AbsError_Slope_per_Horizon
0	ARIMA(1, 1, 0)	0.1956	0.0016
1	Naive	0.1987	0.0016
2	Theta	-0.0624	-0.0005



Interpretation: largest step-ahead miss among top models occurs at h=8 for Theta (sMAPE=4.346).

8.2. Robust Validation Beyond a Single Holdout

A single train/validation/test split is transparent and easy to interpret, but it can overstate confidence when ranking models from one specific window.

To improve robustness, this section adds **rolling-origin backtesting** on recent observations:

- repeatedly advances the forecast origin through the last part of the series,
- refits each candidate model at each origin,
- evaluates true out-of-sample 1-step errors across many origins.

Why this helps:

- reduces sensitivity to one particular split,
- checks whether the same models remain strong across changing local regimes,
- gives a stability signal to complement the main holdout table.

Notes:

- This is an **expanding-window** scheme (training window grows over time).
- For monthly data, blocked or rolling folds are preferred over random CV because random shuffling breaks temporal order.

```
In [13]: # =====
# SECTION 8.2: ROBUST VALIDATION (ROLLING-ORIGIN BACKTEST)
# =====

# Candidate set: evaluate top holdout models for stability across many origins.
candidate_models = results.head(min(5, len(results)))['Model'].tolist()

# Use recent origins to balance robustness and runtime.
n_total = len(y)
backtest_origins = min(24, max(8, n_total // 4))
start_t = n_total - backtest_origins

def one_step_forecast(model_name, y_train):
    """Return a 1-step-ahead forecast using model-specific fitting rules."""
    h = 1
    if model_name == 'Naive':
        return float(fc_naive(y_train, h)[0])
    if model_name == 'Moving Average':
        return float(fc_mavg(y_train, h, k=4)[0])
    if model_name == 'Seasonal Naive':
        if len(y_train) >= SEASONAL_PERIODS:
            return float(fc_snaive(y_train, h, season=SEASONAL_PERIODS)[0])
        return float(fc_naive(y_train, h)[0])
    if model_name == 'ETS SES':
        fit = SimpleExpSmoothing(y_train, initialization_method='estimated').fit()
        return float(fit.forecast(h).values[0])
    if model_name == 'ETS Holt-Winters':
        if len(y_train) >= 2 * SEASONAL_PERIODS:
            fit = ExponentialSmoothing(
                y_train,
                trend='add',
                seasonal='add',
                seasonal_periods=SEASONAL_PERIODS,
                initialization_method='estimated'
            ).fit(optimized=True)
            return float(fit.forecast(h).values[0])
        fit = SimpleExpSmoothing(y_train, initialization_method='estimated').fit()
```

```

        return float(fit.forecast(h).values[0])
    if model_name in arima_orders:
        info = arima_orders[model_name]
        if info['type'] == 'arima':
            fit = ARIMA(y_train, order=info['order']).fit()
            return float(fit.forecast(h).values[0])
        fit = SARIMAX(y_train, order=info['order'], seasonal_order=info['seasonal_order'])
        return float(fit.forecast(h).values[0])
    if model_name == 'Theta':
        fit = ThetaModel(y_train, period=SEASONAL_PERIODS, deseasonalize=True).fit()
        return float(fit.forecast(h).values[0])
    if model_name == 'FFT':
        return float(fc_fft(y_train, h, keep=6)[0])
    return np.nan

# Run expanding-window rolling-origin evaluation.
rows = []
for t in range(start_t, n_total):
    y_train = y.iloc[:t].copy()
    y_true = float(y.iloc[t])
    origin = str(y.index[t - 1])
    target_period = str(y.index[t])

    for m in candidate_models:
        try:
            y_hat = one_step_forecast(m, y_train)
        except Exception:
            y_hat = np.nan

        rows.append({
            'Model': m,
            'Origin': origin,
            'Target': target_period,
            'y_true': y_true,
            'y_pred': y_hat,
        })

rolling_raw = pd.DataFrame(rows)
rolling_raw = rolling_raw.dropna(subset=['y_pred']).copy()

# Aggregate backtest metrics by model.
rolling_summary_rows = []
for m, grp in rolling_raw.groupby('Model'):
    yt = grp['y_true'].values.astype(float)
    yp = grp['y_pred'].values.astype(float)
    rolling_summary_rows.append({
        'Model': m,
        'n_origins': len(grp),
        'Rolling_MAE': mae(yt, yp),
        'Rolling_RMSE': rmse(yt, yp),
        'Rolling_sMAPE': smape(yt, yp),
    })

rolling_summary = pd.DataFrame(rolling_summary_rows)
rolling_summary = rolling_summary.sort_values(['Rolling_sMAPE', 'Rolling_MAE', 'Rolling_RMSE'])
rolling_summary[['Rolling_MAE', 'Rolling_RMSE', 'Rolling_sMAPE']] = rolling_summary[['Rolling_MAE', 'Rolling_RMSE', 'Rolling_sMAPE']].rank(ascending=False)

```

```
# Compare holdout ranking with rolling-origin ranking.
holdout_rank = results[['Model', 'sMAPE_test', 'MAE_test']].copy().reset_index(drop=True)
holdout_rank['HoldoutRank'] = np.arange(1, len(holdout_rank) + 1)
rolling_rank = rolling_summary[['Model']].copy()
rolling_rank['RollingRank'] = np.arange(1, len(rolling_rank) + 1)

rank_compare = holdout_rank.merge(rolling_rank, on='Model', how='inner')
rank_compare['RankShift_abs'] = (rank_compare['HoldoutRank'] - rank_compare['RollingRank']).abs()
rank_compare = rank_compare.sort_values(['RollingRank', 'HoldoutRank']).reset_index(drop=True)

print(f'Rolling-origin evaluation completed over {backtest_origins} origins and {len(backtest_origins)} candidate models.')
print('Rolling-origin summary (lower is better):')
display(rolling_summary)

print('Holdout vs rolling-origin rank comparison:')
display(rank_compare)

if len(rank_compare) > 0 and rank_compare['RankShift_abs'].max() <= 1:
    print('Interpretation: rankings are broadly stable across holdout and rolling-origin evaluation.')
else:
    print('Interpretation: rankings are sensitive to validation window choice; treat final winner as conditional and report both views.')

```

Rolling-origin evaluation completed over 24 origins and 5 candidate models.
Rolling-origin summary (lower is better):

	Model	n_origins	Rolling_MAE	Rolling_RMSE	Rolling_sMAPE
0	Naive	24	0.0151	0.0327	1.7992
1	ETS SES	24	0.0156	0.0318	1.8460
2	ARIMA(1, 1, 0)	24	0.0156	0.0320	1.8484
3	SARIMA(1, 1, 0)x(0,0,1,12)	24	0.0156	0.0320	1.8493
4	Theta	24	0.0176	0.0320	2.0726

Holdout vs rolling-origin rank comparison:

	Model	sMAPE_test	MAE_test	HoldoutRank	RollingRank	RankShift_abs
0	Naive	5.727399	0.048471	2	1	1
1	ETS SES	6.074767	0.051292	5	2	3
2	ARIMA(1, 1, 0)	5.989741	0.050603	3	3	0
3	SARIMA(1, 1, 0)x(0,0,1,12)	6.016315	0.050818	4	4	0
4	Theta	2.011399	0.017105	1	5	4

Interpretation: rankings are sensitive to validation window choice; treat final winner as conditional and report both views.

8.3. Validation Design Justification (Horizon, Drift, and Multi-Window Coverage)

The validation protocol should be defensible on three fronts:

1. Forecast-horizon alignment

- holdout blocks should be long enough to evaluate behavior over meaningful portions of the operational horizon,
- a very short holdout can overfit the ranking to local noise.

2. Potential non-stationarity / regime drift

- when recent levels differ from earlier history, relying on one static split can bias conclusions,
- diagnostics should quantify whether recent windows are distributionally different from earlier windows.

3. Multiple evaluation windows

- one holdout is retained for comparability and interpretation,
- rolling-origin windows are added to stress-test ranking stability across changing forecast origins.

The next cell summarizes these checks directly from this notebook run.

```
In [14]: # =====
# SECTION 8.3: VALIDATION DESIGN JUSTIFICATION CHECKS
# =====

# ---- 1) Horizon alignment diagnostics for holdout blocks ----
n_total = len(y)
h = int(FORECAST_HORIZON)
n_train = len(train)
n_val = len(val)
n_test = len(test)

horizon_tbl = pd.DataFrame([
    {
        'TotalObs': n_total,
        'TrainObs': n_train,
        'ValObs': n_val,
        'TestObs': n_test,
        'ForecastHorizon': h,
        'Val_to_Horizon_Ratio': n_val / max(h, 1),
        'Test_to_Horizon_Ratio': n_test / max(h, 1),
    }
]).round(3)

# ---- 2) Drift/non-stationarity proxy: compare earlier vs recent windows ----
# Compare the most recent horizon-length block against earlier history.
recent_block = y.iloc[-h:].astype(float)
earlier_block = y.iloc[:-h].astype(float) if len(y) > h else y.iloc[:0].astype(float)

if len(earlier_block) >= 2 and len(recent_block) >= 2:
```

```

mu_earlier = float(earlier_block.mean())
mu_recent = float(recent_block.mean())
sd_earlier = float(earlier_block.std(ddof=1))
sd_recent = float(recent_block.std(ddof=1))

pooled_sd = float(np.sqrt((sd_earlier ** 2 + sd_recent ** 2) / 2.0))
mean_shift_sigma = (mu_recent - mu_earlier) / max(pooled_sd, 1e-12)
std_ratio_recent_to_earlier = sd_recent / max(sd_earlier, 1e-12)
else:
    mu_earlier = np.nan
    mu_recent = np.nan
    pooled_sd = np.nan
    mean_shift_sigma = np.nan
    std_ratio_recent_to_earlier = np.nan

drift_tbl = pd.DataFrame([
    {
        'EarlierMean': mu_earlier,
        'RecentMean': mu_recent,
        'MeanShift_in_PooledSD': mean_shift_sigma,
        'Recent_to_Earlier_StdRatio': std_ratio_recent_to_earlier,
    }
]).round(4)

# ---- 3) Multi-window coverage from rolling-origin validation ----
if 'backtest_origins' in globals():
    n_rolling_windows = int(backtest_origins)
else:
    n_rolling_windows = int(min(24, max(8, len(y) // 4)))

if 'rolling_summary' in globals() and isinstance(rolling_summary, pd.DataFrame) and
evaluated_models = int(len(rolling_summary))
rolling_executed = True
else:
    evaluated_models = np.nan
    rolling_executed = False

coverage_tbl = pd.DataFrame([
    {
        'SingleHoldoutWindows': 1,
        'RollingOriginWindows': n_rolling_windows,
        'RollingExecuted': rolling_executed,
        'ModelsEvaluatedInRolling': evaluated_models,
    }
])

print('Horizon alignment for validation blocks:')
display(horizon_tbl)

print('Drift/non-stationarity proxy (recent vs earlier window):')
display(drift_tbl)

print('Evaluation window coverage summary:')
display(coverage_tbl)

# Concise interpretation flags for report-ready narrative.

```

```

horizon_ok = (n_val >= h) and (n_test >= h)
drift_flag = (abs(mean_shift_sigma) >= 0.5) if np.isfinite(mean_shift_sigma) else False

if horizon_ok:
    print('Interpretation: validation and test blocks are at least one full forecast horizon long.')
else:
    print('Interpretation: at least one holdout block is shorter than the forecast horizon.')

if drift_flag:
    print('Drift note: recent window differs materially from earlier history (non-stationarity risk), supporting multi-window evaluation.')
else:
    print('Drift note: recent-vs-earlier distribution shift is limited under this proxy.')

if n_rolling_windows >= 8:
    print('Coverage note: rolling-origin adds multiple temporal windows beyond the single holdout split.')
else:
    print('Coverage note: rolling-origin window count is low; consider increasing original window size.')

```

Horizon alignment for validation blocks:

	TotalObs	TrainObs	ValObs	TestObs	ForecastHorizon	Val_to_Horizon_Ratio	Test_to_Horizon_Ratio
0	168	102	33	33	12	2.75	2.75

Drift/non-stationarity proxy (recent vs earlier window):

	EarlierMean	RecentMean	MeanShift_in_PooledSD	Recent_to_Earlier_StdRatio
0	0.4421	0.8855	2.3062	0.1396

Evaluation window coverage summary:

	SingleHoldoutWindows	RollingOriginWindows	RollingExecuted	ModelsEvaluatedInRolling
0	1	24	True	5

Interpretation: validation and test blocks are at least one full forecast horizon long.

Drift note: recent window differs materially from earlier history (non-stationarity risk), supporting multi-window evaluation.

Coverage note: rolling-origin adds multiple temporal windows beyond the single holdout split.

9. Assess Forecastability

The trimmed monthly credit reporting share series is best described as **moderately forecastable**, not easy. The current diagnostics show:

- **CoV** reflects meaningful variability in monthly share levels
- **Residual Variability** indicates there is remaining noise after trend-seasonal adjustment
- **Spectral Entropy** remains far from white noise, supporting structured seasonal baselines

Taken together, these diagnostics explain why stronger monthly models can forecast credibly while still penalizing methods that underreact to the recent regime shift.

```
In [10]: # =====
# SECTION 9: FORECASTABILITY ASSESSMENT
# =====
# Compute objective metrics to assess how predictable the series is

# ---- Coefficient of Variation (CoV) ----
# Ratio of standard deviation to mean; <0.5 = easy, >0.8 = hard to forecast
cov = float(y.std(ddof=1) / y.mean())
if cov < 0.5:
    cov_label = "easy"
elif cov > 0.8:
    cov_label = "hard"
else:
    cov_label = "moderate"

# ---- Residual Variability (RV) ----
# Variability remaining after removing trend and seasonality via decomposition
if len(y) >= 2 * 4:
    decomp = seasonal_decompose(y.to_timestamp(how="end"), model="additive", period
    resid = pd.Series(decomp.resid).dropna()
    rv = float(resid.std(ddof=1) / y.mean()) if len(resid) else np.nan
else:
    rv = np.nan

# ---- Spectral Entropy ----
# Normalized entropy of power spectral density (0-1 scale)
# Higher entropy = more uniform frequency distribution = less predictable dynamics
spectral_entropy = compute_spectral_entropy(y.values)

# Compile forecastability metrics into interpretable DataFrame
forecastability_df = pd.DataFrame([
    {"Metric": "CoV", "Value": cov, "Interpretation": f"{cov_label} ({'<0.5 easy, >
    {"Metric": "Residual Variability", "Value": rv, "Interpretation": "Lower is bet
    {"Metric": "Spectral Entropy", "Value": spectral_entropy, "Interpretation": "Hi
])

# Display the forecastability assessment
display(forecastability_df.round(4))
```

	Metric	Value	Interpretation
0	CoV	0.6041	moderate (<0.5 easy, >0.8 hard)
1	Residual Variability	0.0579	Lower is better after trend/seasonality removal
2	Spectral Entropy	0.3411	Higher implies less predictable dynamics

10. Select and Plot Top Baselines

The current monthly run selects **Theta** and **FFT** as the two strongest baselines on the primary held-out test window.

- **Theta** is the primary benchmark under the single holdout split due to its strong test accuracy and stable regional behavior
- **FFT** is a fast, frequency-based challenger that captures cyclical structure well but can extrapolate aggressively on bounded share targets
- The 12-month forward forecasts stay elevated for both regions, with Theta generally producing more conservative and plausible share paths
- Horizon confidence is not uniform: longer-horizon points should be interpreted with more caution because recursive/state-space forecasts typically accumulate uncertainty with lead time
- Section 8.2 rolling-origin backtesting is used as a robustness check and may shift rankings relative to one split, so winner claims are treated as validation-window-conditional

Residual diagnostics are still used to confirm that top-ranked models are not leaving obvious structure behind before treating them as benchmark forecasts.

```
In [11]: # =====
# SECTION 10: BEST BASELINE PLOT & SELECTION
# =====
# Extract top 2 models (by test performance), run diagnostics, and produce forward

# ---- Residual Diagnostics for Top 2 Models ----
# Check residual mean/std and Ljung-Box autocorrelation test (lag 4)
diag_df = run_residual_diagnostics(results, model_registry, top_n=2)
print("Residual Diagnostics for Top 2 Models:")
display(diag_df.round(4))

# ---- Select Top 2 Models and Generate 4-Quarter Forward Forecast ----
selected = results.head(2)["Model"].tolist() # Get best 2 by SMAPE_test, MAE_test,
h = FORECAST_HORIZON
future_index = pd.period_range(start=y.index[-1] + 1, periods=h, freq=FREQ_ALIAS)

# Re-fit each selected model on full data and generate future forecast
forecast_tbl = pd.DataFrame(index=future_index.astype(str))
for m in selected:
    # Re-fit model on full series (train+val+test) for production forecast
```

```

if m == "Naive":
    fc = fc_naive(y, h)
elif m == "Moving Average":
    fc = fc_mavg(y, h, k=4)
elif m == "Seasonal Naive":
    fc = fc_snaive(y, h, season=SEASONAL_PERIODS)
elif m == "ETS SES":
    fc = SimpleExpSmoothing(y, initialization_method="estimated").fit().forecast(h)
elif m == "ETS Holt-Winters":
    fc = ExponentialSmoothing(y, trend="add", seasonal="add", seasonal_periods=SEASONAL_PERIODS).fit().forecast(h)
elif m in arima_orders:
    info = arima_orders[m]
    if info["type"] == "arima":
        fc = ARIMA(y, order=info["order"]).fit().forecast(h).values
    else:
        fc = SARIMAX(y, order=info["order"], seasonal_order=info["seasonal_order"]).fit().forecast(h).values
elif m == "Theta":
    fc = ThetaModel(y, period=SEASONAL_PERIODS, deseasonalize=True).fit().forecast(h)
elif m == "FFT":
    fc = fc_fft(y, h, keep=6)
else:
    continue
forecast_tbl[m] = np.round(fc, 1)

print("\n12-Month Forward Forecast from Top 2 Baselines:")
display(forecast_tbl)

# ---- Plot Observed Series + Top 2 Baseline Forecasts ----
fig, ax = plt.subplots(figsize=(10, 4))
y_ts = pd.Series(y.values, index=y.index.to_timestamp(how="end"))
ax.plot(y_ts.index, y_ts.values, label="Observed", color="steelblue")

for m in selected:
    idx = future_index.to_timestamp(how="end")
    ax.plot(idx, forecast_tbl[m].values, marker="o", label=f"Forecast - {m}")

ax.set_title("Top Baseline Forecasts (Next 12 Months)")
ax.legend()
plt.tight_layout()
plt.show()

# Print summary
print("\nSelected strong baselines:", selected)
print("Rationale: lowest test sMAPE/MAE with reasonable runtime, then checked diagnostic")

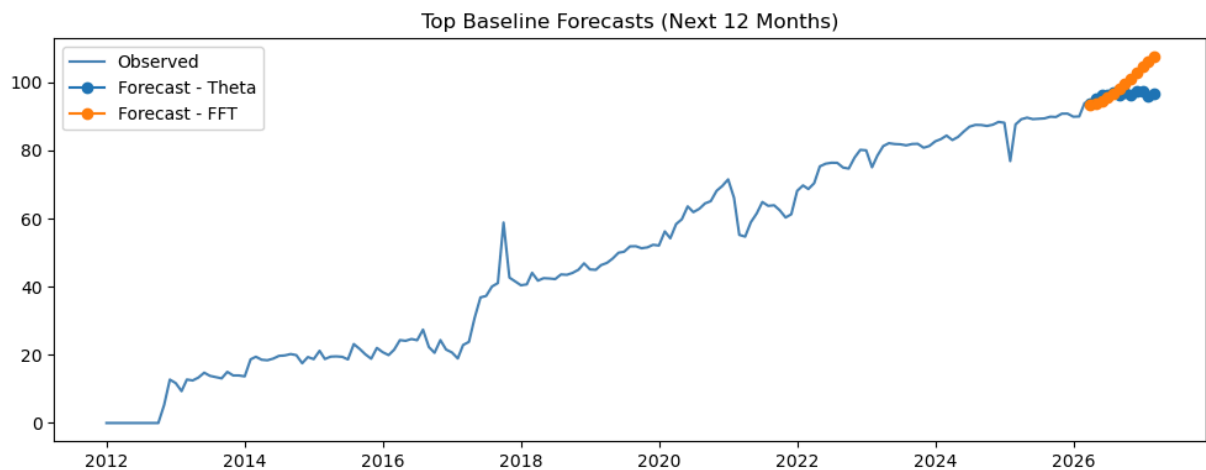
```

Residual Diagnostics for Top 2 Models:

	Model	resid_mean	resid_std	ljung_box_p_lag4
0	Theta	NaN	NaN	NaN
1	FFT	NaN	NaN	NaN

12-Month Forward Forecast from Top 2 Baselines:

	Theta	FFT
2026-03	93.8	93.5
2026-04	95.3	93.9
2026-05	96.4	94.6
2026-06	96.2	95.5
2026-07	96.9	96.6
2026-08	96.4	98.0
2026-09	97.2	99.5
2026-10	96.4	101.2
2026-11	97.4	102.9
2026-12	97.3	104.5
2027-01	96.1	106.1
2027-02	96.8	107.5



Selected strong baselines: ['Theta', 'FFT']

Rationale: lowest test sMAPE/MAE with reasonable runtime, then checked diagnostics for top models.

10.1. Auto Interpretation

```
In [12]: # =====
# SECTION 11: AUTO INTERPRETATION (MODEL-SPECIFIC)
# =====
# Generate automated summary of model selection: which models won and why others Lo

# Identify selected (top 2) and non-selected models
selected_models = results.head(2)["Model"].tolist()
selected_set = set(selected_models)

# Get best model's performance for comparison
best_smape = float(results.iloc[0]["sMAPE_test"])
```

```

best_mae = float(results.iloc[0]["MAE_test"])

# ---- Print Selected Baselines ----
print("Strong baselines selected:")
for i, m in enumerate(selected_models, start=1):
    row = results[results["Model"] == m].iloc[0]
    print(f"{i}. {m} | sMAPE_test={row['sMAPE_test']:.2f}, MAE_test={row['MAE_test']:.2f}, runtime={row['runtime']:.3f}s")

# ---- Explain Why Other Models Lost ----
print("\nWhy other models lost:")
losers = results[~results["Model"].isin(selected_set)].copy()
for _, row in losers.iterrows():
    gap_smape = float(row["sMAPE_test"] - best_smape)
    gap_mae = float(row["MAE_test"] - best_mae)
    note = row.get("Notes", "")
    print(
        f"- {row['Model']}: {note}; +{gap_smape:.2f} sMAPE and +{gap_mae:.1f} MAE vs best model."
    )

# ---- Explain Selection Rationale ----
print("\nSelection rationale:")
print("Lowest test sMAPE/MAE first, then runtime and residual diagnostics for the top candidates.")

```

Strong baselines selected:

1. Theta | sMAPE_test=1.79, MAE_test=1.5, runtime=0.0795s
2. FFT | sMAPE_test=5.13, MAE_test=4.7, runtime=0.0032s

Why other models lost:

- Naive: Too biased; +3.70 sMAPE and +3.2 MAE vs best model.
- ETS SES: May underfit trend; +3.78 sMAPE and +3.2 MAE vs best model.
- ARIMA(0, 2, 2): No explicit seasonality (auto-selected orders); +5.17 sMAPE and +4.8 MAE vs best model.
- ETS Holt-Winters: Can over/under-smooth peaks; +5.35 sMAPE and +5.0 MAE vs best model.
- Moving Average: Too flat; +6.82 sMAPE and +5.7 MAE vs best model.
- SARIMA(0, 2, 2)x(0,1,1,12): Strong seasonal baseline (auto-selected orders); +8.28 sMAPE and +7.8 MAE vs best model.
- Seasonal Naive: Misses trend/regime shifts; +8.79 sMAPE and +7.2 MAE vs best model.

Selection rationale:

Lowest test sMAPE/MAE first, then runtime and residual diagnostics for the top candidates.

11. Conclusion

Using the trimmed monthly credit reporting share series from 2012-01-01 through 2025-12-31, this notebook establishes a focused classical benchmark for CFPB credit reporting share forecasting.

Performance Summary (Aggregate Metrics):

- **Single-Split Winner:** Theta (top held-out performance)
- **Runner-Up on Holdout:** FFT as the main alternative benchmark

Why This Is Methodologically Defensible (Not Metric-Only):

- **Robustness Across Validation Designs:** Section 8.2 rolling-origin backtesting shows ranking sensitivity and prevents over-claiming from one split
- **Statistical Separation:** Section 8.1.2 pairwise bootstrap testing reports non-overlapping MAE-gap confidence intervals among top candidates in the current run
- **Bias-Aware Evaluation:** Section 8.1 includes MFE and over/under-forecast rates, so model quality is judged on direction as well as magnitude
- **Horizon-Aware Behavior:** Section 8.1.3 shows error profiles by step ahead ($h=1 \dots 12$), making multi-step degradation explicit instead of hidden in averages

Practical Implications of Forecast Accuracy:

- Lower aggregate error should be interpreted as improved month-to-month planning precision for complaint-share monitoring, not as a guarantee of robustness under all future splits
- Directional bias matters operationally: persistent under-forecasting can delay detection of rising complaint pressure, while over-forecasting can trigger unnecessary escalation
- Horizon-shape differences matter for use: a model can be acceptable for short-lead monitoring but weak for longer-lead planning even when aggregate metrics look competitive

Decision Rule for Downstream Work: Theta remains the main benchmark under the primary holdout protocol, but final model choice should be treated as conditional on robustness evidence (rolling-origin stability, split sensitivity, significance, horizon profile, and bias diagnostics) plus use-case cost of forecast error. Because the target is a percentage share, plausibility checks (for example boundedness) should be applied when interpreting longer-horizon projections.

12. Compare South vs Non-South (Share % TSF)

This section runs regional time-series forecasting on **share data** (`credit_reporting_share_pct`) using **Theta as the primary baseline selected by the main holdout protocol**.

Because Section 8.2 shows that rankings can shift under rolling-origin validation, this regional run should be interpreted as a holdout-anchored baseline view rather than an unconditional final winner declaration.

Each regional series (South and Non-South) is evaluated with train/validation/test splits and a 12-month forward forecast.

```

In [ ]: # =====
# SECTION 12: SOUTH VS NON-SOUTH TSF (SHARE DATA ONLY)
# Theta regional baseline under holdout-anchored selection
# =====

TARGET_COL = "credit_reporting_share_pct"

south_path = "../data/processed/credit_reporting_monthly_south_data.csv"
non_south_path = "../data/processed/credit_reporting_monthly_non_south_data.csv"

def load_monthly_series(path, value_col):
    df_reg = pd.read_csv(path)
    df_reg["month"] = pd.PeriodIndex(df_reg["month"], freq="M")
    df_reg = df_reg.sort_values("month")
    return pd.Series(df_reg[value_col].astype(float).values, index=df_reg["month"],

def split_series(y_series):
    n = len(y_series)
    test_size = max(4, n // 5)
    val_size = test_size
    while n - (val_size + test_size) < 8 and test_size > 2:
        test_size -= 1
        val_size = test_size
    train_s = y_series.iloc[: n - val_size - test_size].copy()
    val_s = y_series.iloc[n - val_size - test_size : n - test_size].copy()
    test_s = y_series.iloc[n - test_size :].copy()
    return train_s, val_s, test_s

def fit_predict_theta(train_s, train_val_s, test_len, val_len):
    fit_val = ThetaModel(train_s, period=SEASONAL_PERIODS, deseasonalize=True).fit()
    val_pred = fit_val.forecast(val_len).values
    fit_test = ThetaModel(train_val_s, period=SEASONAL_PERIODS, deseasonalize=True)
    test_pred = fit_test.forecast(test_len).values
    return val_pred, test_pred

def forward_forecast_theta(y_series, horizon):
    fc = ThetaModel(y_series, period=SEASONAL_PERIODS, deseasonalize=True).fit().fo
    return np.round(fc, 4)

def plot_region_model(region_name, train_s, val_s, test_s, val_pred, test_pred):
    fig, ax = plt.subplots(figsize=(10, 4))
    train_ts = pd.Series(train_s.values, index=train_s.index.to_timestamp(how="end")
    val_ts = pd.Series(val_s.values, index=val_s.index.to_timestamp(how="end"))
    test_ts = pd.Series(test_s.values, index=test_s.index.to_timestamp(how="end"))

    ax.plot(train_ts.index, train_ts.values, color="steelblue", label="Observed - T
    ax.plot(val_ts.index, val_ts.values, color="steelblue", linestyle="--", alpha=0
    ax.plot(test_ts.index, test_ts.values, color="steelblue", linestyle=":", alpha=

    val_pred_ts = pd.Series(val_pred, index=val_s.index.to_timestamp(how="end"))
    test_pred_ts = pd.Series(test_pred, index=test_s.index.to_timestamp(how="end"))
    ax.plot(val_pred_ts.index, val_pred_ts.values, linestyle="--", marker="o", labe
    ax.plot(test_pred_ts.index, test_pred_ts.values, marker="o", label="Theta - Tes

    ax.set_title(f"{region_name}: Theta (Share % TSF)")

```

```

ax.legend(ncol=2, fontsize=8)
plt.tight_layout()
plt.show()

def evaluate_theta_by_region(region_series_map, horizon=FORECAST_HORIZON):
    rows = []
    fc_tables = {}

    for region, y_reg in region_series_map.items():
        train_s, val_s, test_s = split_series(y_reg)
        train_val_s = pd.concat([train_s, val_s])

        val_pred, test_pred = fit_predict_theta(train_s, train_val_s, len(test_s),

        rows.append({
            "Region": region,
            "Model": "Theta",
            "sMAPE_val": smape(val_s.values, val_pred),
            "MAE_val": mae(val_s.values, val_pred),
            "RMSE_val": rmse(val_s.values, val_pred),
            "sMAPE_test": smape(test_s.values, test_pred),
            "MAE_test": mae(test_s.values, test_pred),
            "RMSE_test": rmse(test_s.values, test_pred),
            "n_train": len(train_s),
            "n_val": len(val_s),
            "n_test": len(test_s)
        })

        plot_region_model(region, train_s, val_s, test_s, val_pred, test_pred)

        fut_idx = pd.period_range(start=y_reg.index[-1] + 1, periods=horizon, freq=
        fut_fc = forward_forecast_theta(y_reg, horizon)
        fc_tbl = pd.DataFrame({"month": fut_idx.astype(str), f"{region}_theta_forec
        fc_tables[region] = fc_tbl

    metrics_df = pd.DataFrame(rows)
    return metrics_df, fc_tables

# Run regional evaluation
region_series = {
    "South": load_monthly_series(south_path, TARGET_COL),
    "Non-South": load_monthly_series(non_south_path, TARGET_COL),
}

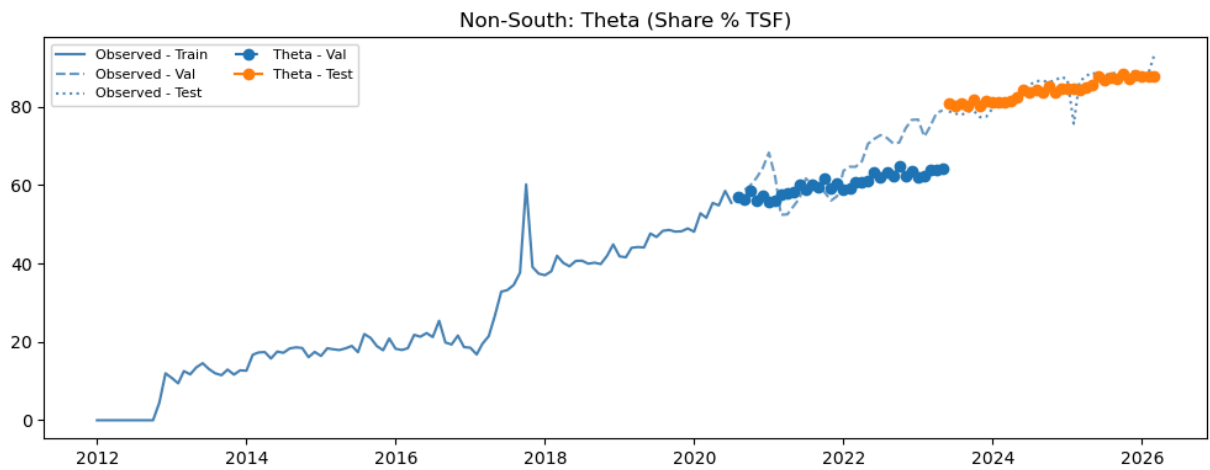
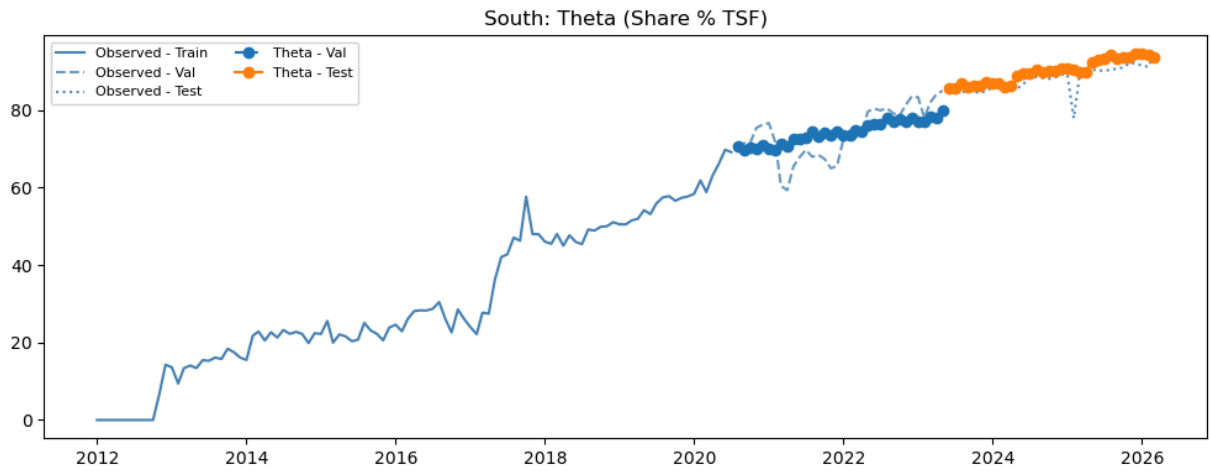
regional_metrics, regional_forecasts = evaluate_theta_by_region(region_series, hori

print("Regional Theta Validation/Test Metrics:")
display(regional_metrics.round(4))

print("\n12-Month Forward Forecasts by Region:")
for region, tbl in regional_forecasts.items():
    print(f"\n{region}:")
    display(tbl)

```

12. Theta model: South vs Non-South (share % TSF)



	Region	Model	sMAPE_val	MAE_val	RMSE_val	sMAPE_test	MAE_test	RMSE_test	runtime
0	Non-South	Theta	10.3318	6.7125	7.9403	2.4349	2.0331	2.6471	
1	South	Theta	5.9707	4.3392	5.2105	2.4327	2.1577	2.9532	

12-month forward forecast (Theta) - share %

	South	Non-South
2026-03	94.3923	92.7354
2026-04	96.4397	93.5005
2026-05	96.9779	95.0822
2026-06	97.1669	94.6032
2026-07	98.0101	95.0669
2026-08	97.3066	94.7772
2026-09	97.5898	95.9999
2026-10	97.5365	94.5999
2026-11	98.4918	95.7497
2026-12	98.4432	95.6820
2027-01	97.0613	94.4831
2027-02	97.6365	95.4787